Practical Use Cases

Seven tips, one forecasting experiment

Speaker: Dermot O’Brien

Seven tips for working with AI

TIP 1

Learn how to program, use your domain knowledge

AI multiplies what you already know. Bring expertise; AI brings throughput.

TIP 2

Be as specific as humanly possible

Most prompts don't give enough context. Paste in docs, references, `llms.txt`, screenshots. Or draft a rough prompt and ask AI to rewrite it *"using LLM best practices."*

TIP 3

Smaller tasks beat big ones

Break problems down. If you can't break it down, you don't understand it yet.

TIP 4

Tell the AI what you *don't* want

Three-section pattern: **TASK / BACKGROUND / DO NOT**.

TIP 5

Tell AI to remember

`AGENTS.md` (context) + skills (procedures) in `.claude/`. Ask the agent to draft v1.

The experiment

- **Task**: forecast 168 hours of hourly German electricity load, 2020-01-01 to 2020-01-07.
- **Data**: Open Power System Data (OPSD) / ENTSO-E, 2015–2020, hourly.
- **Agent**: Claude Code, Opus 4.7 — same agent in every run.
- **Code, prompts, and full results**: github.com/DermotOBrien-EC/LLM_coding_practical_example_JRC_LEGENT_presentation_15_05_26
- **Three prompts, three workspace setups:**

Level 1 — beginner

"Forecast first week of January 2020 German hourly electricity load."

10 words · no AGENTS.md

Level 2 — average

Names data, horizon, libraries. Asks for plot + accuracy.

46 words · 7-line AGENTS.md

Level 3 — research-grade

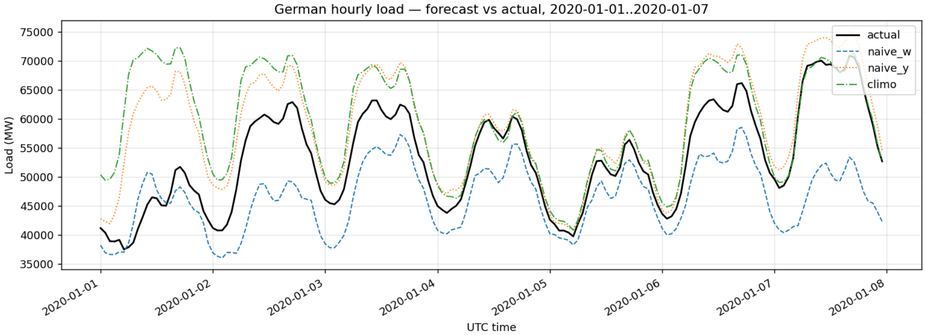
Structured TASK / BACKGROUND / DO NOT. 6-model bake-off, validation, intervals.

1,673 words · 113-line AGENTS.md

OPSD time series: https://data.open-power-system-data.org/time_series/

Level 1: what 10 words gets you

"Forecast first week of January 2020 German hourly electricity load."



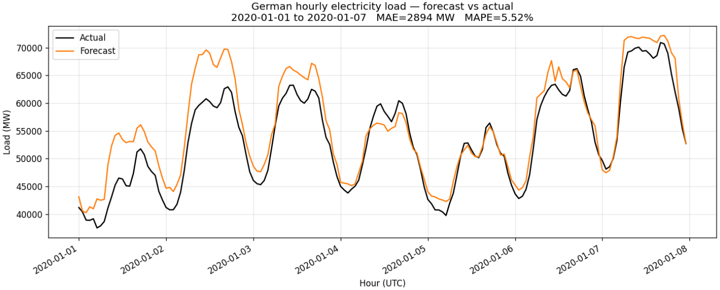
Three baselines compared; L1 picked the best of three: ***seasonal-naive (lag-364d), MAPE 10.76%.***

No validation set. No prediction intervals. No methodology document.

Level 2: average prompt + thin workspace

“Build me a Python script that forecasts German hourly electricity load for the first week of January 2020. Train on the historical data, plot it, and tell me how accurate it was.”

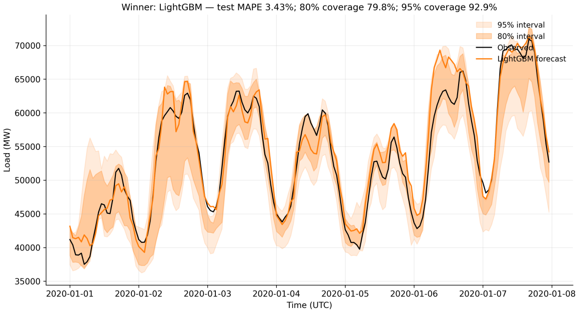
AGENTS.md: Python project. opsd_de_load.csv has hourly German load. Use pandas. Plot with matplotlib.



***GradientBoostingRegressor**, engineered features. **MAPE 5.52%.** Still no held-out validation, no intervals, no methods document.*

Level 3: research-grade prompt

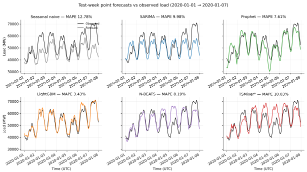
Prompt followed the TASK / BACKGROUND / DO NOT pattern (Tip 4). 1,673 words · 113-line AGENTS.md · 6-model bake-off with held-out validation and calibrated intervals.



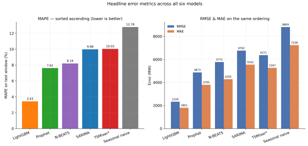
LightGBM won a six-model bake-off. **MAPE 3.43%.** 80% PI covered **79.8%** · 95% PI covered **92.9%** — essentially nominal.

L2 might have got lucky picking a decent model.
L3 didn't have to be lucky — it searched, validated, calibrated, and documented.

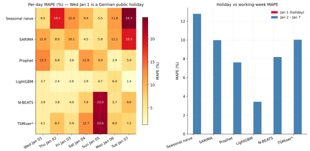
Some of what L3 actually produced



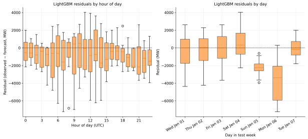
Six-model comparison



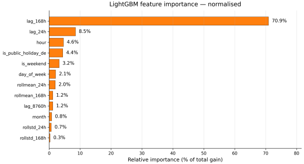
Scoreboard — MAPE, RMSE, MAE



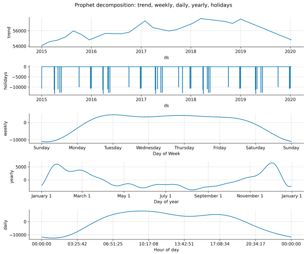
Per-day MAPE heatmap



Winner residuals by hour and day



Feature importance (normalised)



Prophet trend / seasonal decomposition

L1 produced 1 figure. L3 produced 8.

Seven tips for working with AI (cont.)

TIP 6

MCPs extend what AI can do

Plug-in tools at runtime. Context7 (docs), Hugging Face (papers), GitHub (repos).

TIP 7

Always give AI a way to verify its work

Tests, held-out validation, calibrated intervals. Have AI generate them *and* read what it wrote — AI-written tests can "pass" while testing the wrong thing.

Closing thesis

AI amplifies the habits you already have.

Let it type for you — not think for you.

Bring good engineering habits and AI multiplies them.

Skip tests, skip docs, skip the "what could go wrong" question — and AI amplifies *that* instead.